



The sciences have identified restrictions on

when (src) {  **src.balance += 10; }**

when (dst) {  **dst.balance += 10; }**



when(dst) {  **print(dst.balance) }**

when(src) {  **print(src.balance) }**

src=0, dst=0



src=0, dst=10



src=10, dst=0



src=10, dst=10



Order

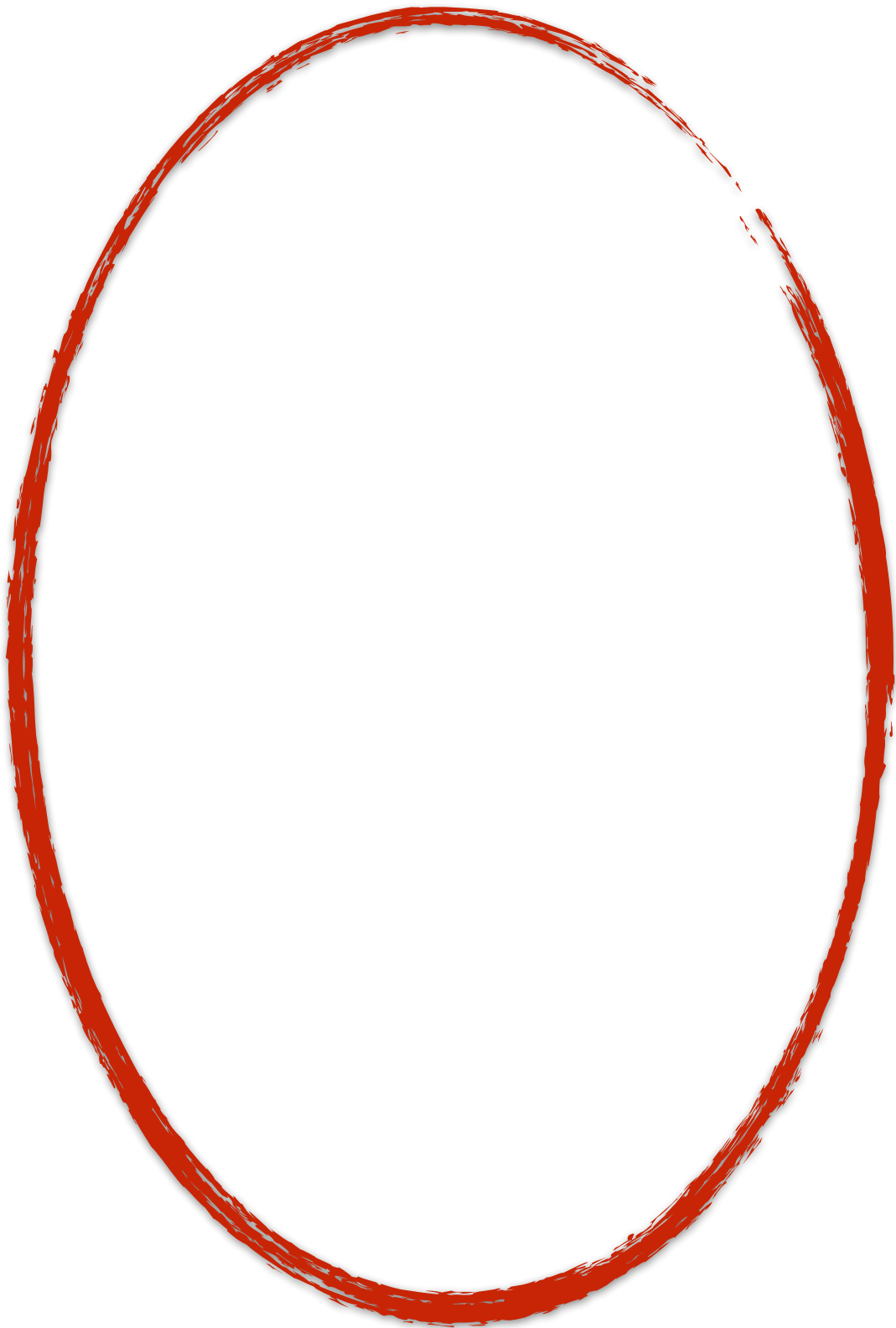
Permitted





updatepath(src then dst)

Diagnostic path (dst then src)



The semantics have incidental restrictions on executions

Update path (src then dst)

when(src) { A src.balance += 10; }

when(dst) { B dst.balance += 10; }

Diagnostic path (dst then src)

when(dst) { C print(dst.balance) }

when(src) { D print(src.balance) }

	src=0, dst=0	src=0, dst=10	src=10, dst=0	src=10, dst=10
Order	C B D A	B C D A	A C B D	A B C D
	C D A B	B D A C	A C D B	A B D C
	C D B A	B D C A	A D C B	A D B C
	D A C B	D A B C	C A B D	B A C D
	D C A B	D A B C	C A D B	B A D C
	D C B A	D B C A	C B A D	B C A D
Permitted	✓	⚠	✓	✓

There are restrictions we want to keep

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    }  
  }  
}
```

A nested behaviour is spawned synchronously,
but can runs asynchronously of it's parent